

PMOS™

PATTERN RECOVERY FIELD GUIDE

Consultant Edition

How to recognise, diagnose and recover the five behavioral patterns
that cause ERP and transformation programs to fail

*Case studies · the phrases that signal each pattern · CIO/Sponsor/PM/Team views ·
maturity ladders · industry variants · cost of inaction · pattern interactions*

Karan Singh Rawat · ERP Delivery Manager · 17 years · 15+ enterprise implementations

How to use this guide

This is the consultant's-eye view of the five PMOS patterns. It is built for recognition and judgement, not process. Each pattern is written so that if you have run real ERP programs, you will recognise your own situation on nearly every page.

For each pattern you get: the **phrases** that signal it in a meeting; two or three **real recovery case studies** across SAP, Oracle and Dynamics; the same pattern seen from the **CIO, sponsor, PM and team**; a **maturity ladder** to place your program; the **cost of inaction**; the **consultant observations** teams usually miss; and how each pattern **breeds the others**.

The order is deliberate. The two most go-live-critical patterns come first, then governance and resistance, and Builder Culture last — because it is the pattern that governs whether the other four stay fixed.

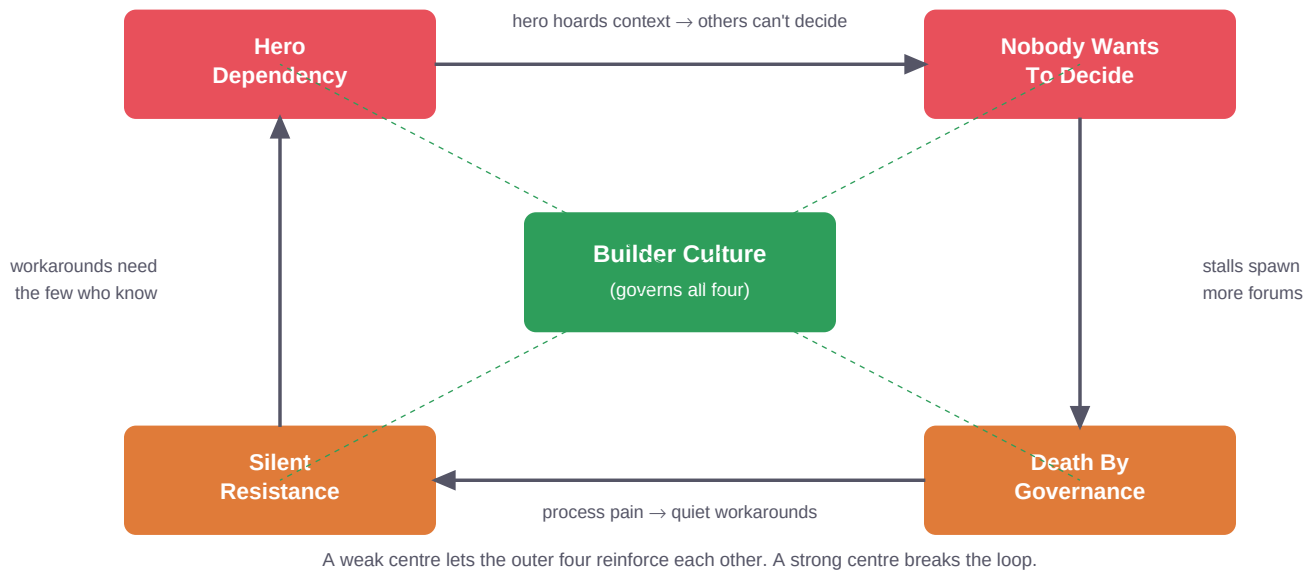
Pairs with your Command Center. This guide is the judgement layer; the PMOS Recovery Command Center workbook is where you run and track the work. The guide tells you how to think about each pattern; the workbook scores it, links its risks, decisions and actions, and projects your recovery. Use them together.

The PMOS frameworks

Six proprietary frameworks sit behind this guide. They are the mental models a recovery lead uses to make sense of a failing program — read them first; they recur throughout the patterns that follow.

PMOS PATTERN RELATIONSHIP MAP

how the five patterns drive one another



PMOS RECOVERY SEQUENCE

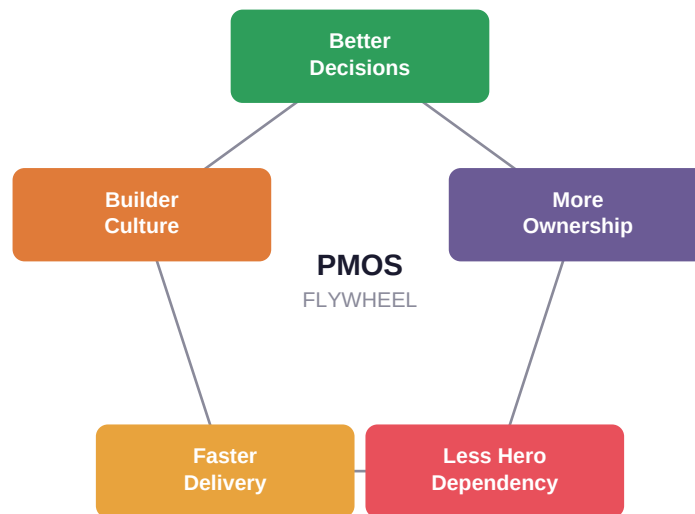
the order that works — and why



Order matters: doing the right things in the wrong sequence is how recoveries stall.

PMOS RECOVERY FLYWHEEL

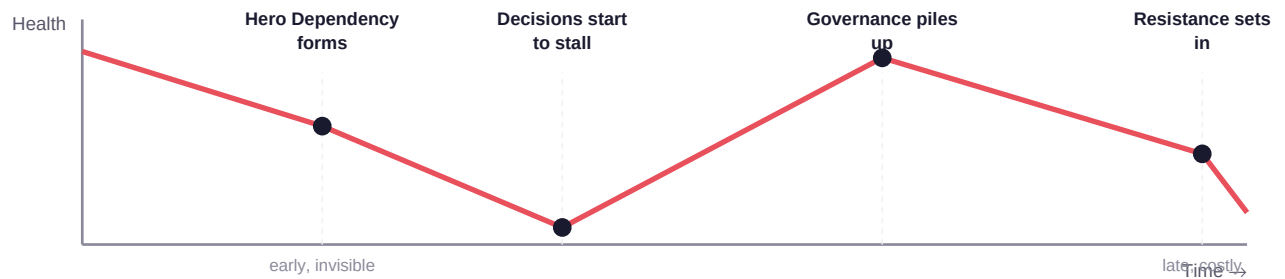
why recovery accelerates once it starts



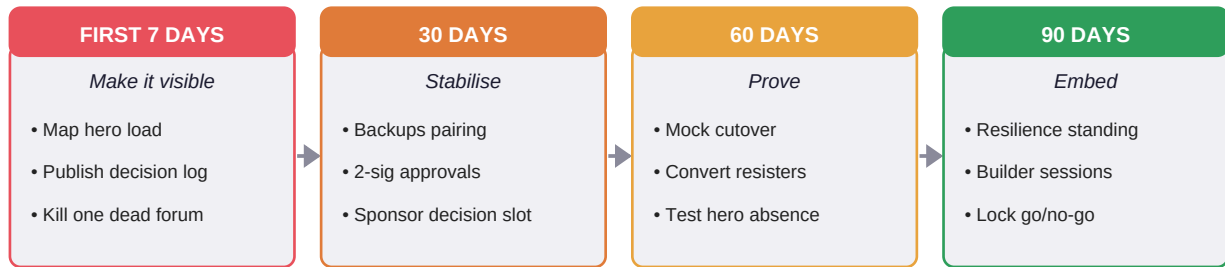
Each turn makes the next easier. Recovery compounds once the wheel moves.

PATTERN EVOLUTION TIMELINE

how programs deteriorate when patterns go unspotted



Patterns appear in sequence. The earlier you catch the curve, the cheaper the recovery.



Visible → Stable → Proven → Embedded. The arc every PMOS recovery follows.

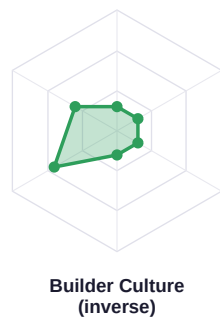
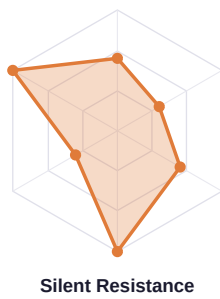
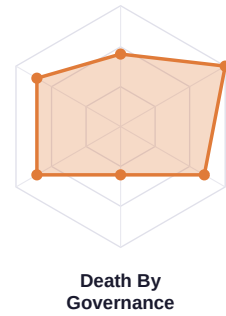
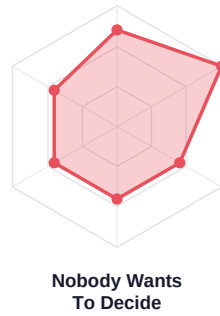
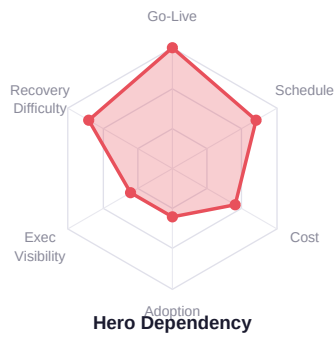
PMOS MATURITY JOURNEY

Critical → Resilient

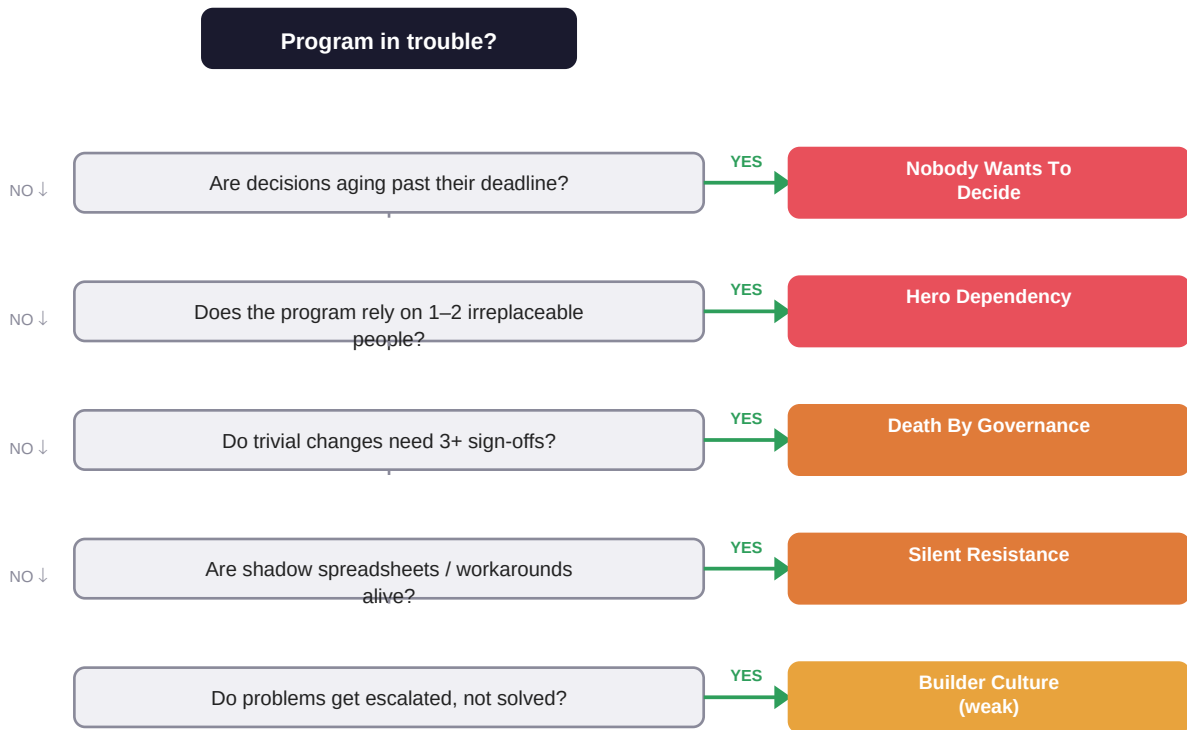
The recovery journey — each stage has a defined behavioural signature in the field guide.



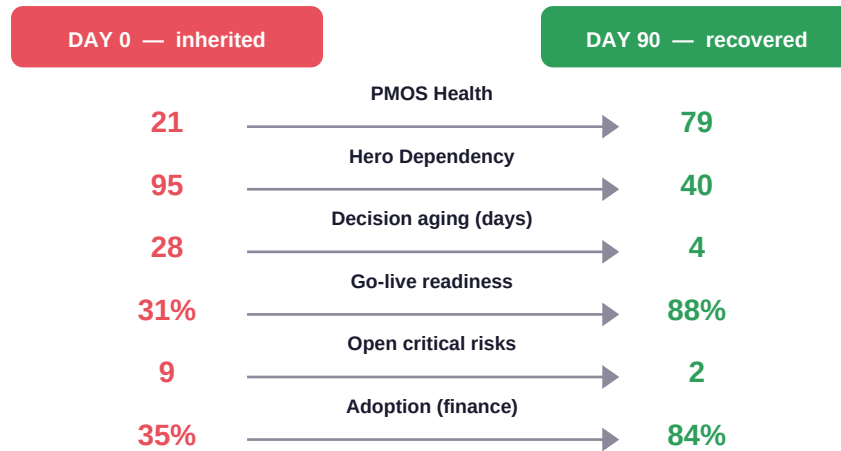
Most programs enter recovery at Critical or Elevated. The goal is not Healthy — it's Resilient.



Each pattern has a distinct risk fingerprint — compare the shapes to see where each one hurts most.



Work top to bottom. The first YES is your primary pattern — the rest compound it.



Project Atlas — a real recovery. Same program, 90 days apart. This is what the system delivers.

Hero Dependency™

SCORE 90 • CRITICAL

The whole program is balanced on one or two people who can't be replaced and won't take leave. Everyone knows who they are. Nobody has done anything about it — because while they're there, it works.

Phrases that tell you it's here

PHRASES YOU HEAR IN THE ROOM (each one is a tell)

"Let's just ask Raj, he'll know." → One name is the answer to every hard question — that's the dependency, spoken aloud.

"We can't start that until Maria's back from leave." → Delivery is gated on a single person's calendar.

"Don't worry, I'll handle the cutover myself." → The hero is volunteering for the single point of failure, and everyone's relieved.

"It's quicker if I just do it than explain it." → The exact sentence that builds the dependency, repeated until it's structural.

"Honestly, I haven't taken a day off since UAT started." → Said with pride. It's a red flag, not a badge.

What it looks like in the wild

CASE 1 Discrete manufacturer, SAP S/4 — the pricing-config hostage

Situation: A single external consultant had built every pricing condition record and variant configuration. 9 weeks to go-live.

The tell: When he booked a two-week holiday, the PM quietly asked him to cancel it. He did. Everyone exhaled. Nobody saw the problem.

What happened: He resigned three weeks before go-live for a better offer. No runbook existed. Go-live slipped 7 weeks while a replacement reverse-engineered the config from the system.

The lesson: The dependency was visible for months. The trigger to act never came because the hero kept absorbing it — right up until he didn't.

CASE 2 Pharma distributor, Oracle — the integration whisperer

Situation: One internal architect understood the middleware between the WMS, the ERP and the serialisation system. It was genuinely complex.

The tell: Hypercare planning had her name in every escalation path, all shifts. 'She's fine, she'll cover it.'

What happened: She burned out in week 2 of hypercare and went on sick leave. Batch-release integrations failed with no one able to triage. Product sat in quarantine; a regulated shipment window was missed.

The lesson: Hero Dependency doesn't always end in resignation. Sometimes the hero is simply human, and the program never planned for that.

CASE 3 Retail chain, Dynamics — the silent two-person bus factor

Situation: Two developers held all store-replenishment logic. They worked well together, which hid the risk — to outsiders it looked like a team.

The tell: Code reviews were them reviewing each other. No third person had ever touched the replenishment engine.

What happened: Both were poached by the same systems integrator in the same month. The 'team' evaporated. Replenishment defects in the first peak season cost an estimated £2.1M in lost margin and stockouts.

The lesson: A two-person hero team feels safer than a one-person one. It isn't — it's the same risk wearing a disguise.

How it shows up by industry

Industry	How the pattern shows up here specifically
Manufacturing	One person owns variant configuration / pricing conditions / production version logic — the most config-heavy, least-documented corner of the build.
Pharma / Life Sci	The validation and serialisation specialist becomes irreplaceable because regulatory knowledge plus system knowledge is rare in one head.
Professional Services	The revenue-recognition / project-accounting expert is the bottleneck — complex rules, one interpreter, no backup.
Retail	Replenishment, allocation and promotions logic concentrate in one or two developers who understand the seasonal peaks no document captures.

The same pattern, seen from four chairs

Seen from...	What this pattern looks like to them
CIO	Sees a green delivery RAG and a confident lead. The dependency is invisible at their altitude until it becomes an incident. They are the last to know.
Sponsor	Likes the hero — they get things done, they're reassuring in steering. The sponsor unintentionally rewards the behaviour that's building the risk.
PM	Knows exactly who the heroes are and is privately terrified, but raising it feels like admitting the plan is fragile. So it stays unsaid.
The team	Has learned not to step on the hero's turf. Initiative was quietly discouraged, so they wait — which deepens the dependency further.

The organisational roots (not the individuals)

- **Incentives:** the hero is rewarded — recognition, bonus, status — for being indispensable. The org is paying for the exact behaviour that endangers it.
- **Vendor behaviour:** SIs deliberately concentrate knowledge to protect day-rate revenue. A documented, transferable build is a commercial threat to them.
- **Governance:** no resilience gate exists. 'Two named owners per critical role' is nobody's acceptance criterion, so it never gets checked.
- **Sponsorship:** the sponsor optimises for this quarter's delivery, not next quarter's resilience, and the hero lets them — because being needed feels safe.

Maturity ladder — where is your program?

Level	What it looks like in practice
1 • Critical	Single individuals are the only point of delivery for critical areas. No backups, no runbooks, heroes haven't taken leave in months.
2 • Fragile	Backups exist on paper but have never executed solo. Knowledge transfer is 'planned'. One absence still causes a stall.
3 • Aware	Hero load is mapped and named. Pairing has started on the highest-risk areas. The org can describe its dependencies even if it hasn't closed them.
4 • Resilient	Every critical role has two competent names. Absence has been tested without a stall. Runbooks are executed, not just written.

Cost of inaction & go-live impact

If you do nothing...	Typical consequence	Rough cost signal
Hero unavailable in cutover week	Go-live slips while knowledge is reconstructed	6–10 week delay
Hero resigns pre-go-live	Reverse-engineering the undocumented build	£0.5–2M + timeline
Hero burns out in hypercare	Support function collapses; incidents unresolved	Adoption + reputation hit
Dependency never addressed	Permanent key-person risk into BAU / support	Ongoing, compounding

What an experienced recovery lead sees that the team doesn't

What the team usually misses: The hero is not the risk — the system that created them is. If you remove this hero without changing the incentives and governance, the program simply crowns a new one within a month. Fix the conditions, not just the person.

What the team usually misses: The most dangerous hero is the one everyone trusts most, because their competence is exactly what stops anyone questioning the concentration of risk. Look hardest where you feel most reassured.

What the team usually misses: Heroes resist backups not from ego but from identity — being indispensable is how they've always been valued. Frame transfer as a promotion in trust ('I want you leading this, not carrying it'), never as a redundancy.

How Hero Dependency breeds the other patterns

How this pattern breeds others: It manufactures **Nobody Wants To Decide**: when one person holds all the context, everyone else defers decisions to them — and when they're unavailable, decisions simply stop. It also masks **Death By Governance**: the hero quietly routes around broken process, so the organisation never feels the pain that would force it to fix governance. And it suppresses **Builder Culture**: a team trained to defer to the hero never develops the muscle to solve problems itself. Break the hero dependency and you often expose how weak the decision-making and problem-solving culture underneath really was.

HERO DEPENDENCY BUS-FACTOR MAP

knowledge concentration + backup coverage — worked example

Area	Config L. Arch	Integration Int Dev	Data Data Ld	Cutover Cut Mgr	Hypercare Svc Own
Backup	none	none	J. Snr	none	partial

BUS FACTOR: 3 of 5 critical areas have a single owner and no backup.

Any one of them off in cutover week moves the date.

Red = single owner, no backup. The backup row is the recovery target: turn every red green before cutover.

Nobody Wants To Decide™

SCORE 78 · CRITICAL

Decisions don't get made; they get discussed. Everyone is consulted, no one is accountable, and the same question returns meeting after meeting until the deadline makes the decision for you — badly.

Phrases that tell you it's here

PHRASES YOU HEAR IN THE ROOM (each one is a tell)

"Let's take this offline." → The graveyard of decisions. 'Offline' means 'nowhere', and everyone knows it.

"I think we need a bit more data before we commit." → Sometimes true. Usually a socially acceptable way to not decide.

"Let's park that and come back to it." → The car park is full. Nothing ever comes back from it.

"We should probably socialise this more widely first." → Consultation as avoidance — widening the circle so no one has to own it.

"Can we get alignment from all the stakeholders on this?" → 'Alignment' here means unanimous consent, which means a veto for everyone and a decision from no one.

What it looks like in the wild

CASE 1 Industrial manufacturer, SAP — the chart-of-accounts that ate six weeks

Situation: A 4-segment vs 6-segment chart-of-accounts decision sat open. Finance, the SI, and group reporting each had a view. None owned the call.

The tell: It appeared on seven consecutive design-authority agendas, each ending 'let's get finance and group aligned'.

What happened: The build team proceeded on an assumption to avoid stalling. When the decision finally landed — different from the assumption — it triggered six weeks of rework across every finance object.

The lesson: A decision not made is still a decision; it just gets made by default, late, and expensively. The cost wasn't the decision — it was the delay and the rework the delay caused.

CASE 2 Healthcare provider, Oracle — the go/no-go nobody owned

Situation: Three weeks from cutover, the go/no-go criteria were 'being finalised'. No single person owned the go decision — it was 'the steering committee's'.

The tell: When asked 'who says go?', the answer was a room, not a name.

What happened: On the night, two criteria were amber. With no owner empowered to make the call, the committee defaulted to 'proceed' to avoid being the one to stop it. Hypercare became a five-month firefight.

The lesson: The biggest decision on the program had no owner — so it was made by social pressure at midnight, which is the worst possible way to make it.

CASE 3 Services firm, Dynamics — the localisation stalemate

Situation: Global template vs country localisation for 14 markets. The sponsor delegated it to a working group; the working group escalated it back as 'too strategic for us'.

The tell: The decision bounced up and down the hierarchy four times in eight weeks — a perfect stalemate of mutual deferral.

What happened: Each market built to its own assumption while waiting. Integration testing collapsed under 14 incompatible interpretations. The template was effectively abandoned, destroying the business case for a single ERP.

The lesson: When a decision bounces between levels, it's not too hard to make — it's that no level has been given the authority and the mandate to own it.

How it shows up by industry

Industry	How the pattern shows up here specifically
Manufacturing	Foundational design decisions (COA, costing approach, plant/storage-location structure) stay open into build, because they're cross-functional and no one function will own them.
Pharma / Life Sci	Validation and compliance decisions stall because no one wants to be accountable for a regulatory interpretation that could later be wrong.
Professional Services	Revenue-recognition and project-structure decisions circle endlessly between finance and delivery, each deferring to the other's domain.
Retail	Promotion, pricing and channel-priority rules stay unsettled because they cross merchandising, finance and operations — three owners, so no owner.

The same pattern, seen from four chairs

Seen from...	What this pattern looks like to them
CIO	Sees a program 'in design' far longer than planned, but reads it as thoroughness, not paralysis. Rarely sees that the delay is decisional, not technical.
Sponsor	Is often the bottleneck without knowing it — delegating decisions back down that only they have the authority to make, then wondering why nothing moves.
PM	Can see exactly which decisions are stuck and what they're blocking, but lacks the authority to force closure and the cover to escalate hard.
The team	Learns that decisions don't stick, so they hedge — building to assumptions, keeping options open, which multiplies rework when the decision finally lands.

The organisational roots (not the individuals)

- **Incentives:** being wrong is punished harder than being slow. In that culture, not-deciding is the rational, career-safe choice — so people don't.
- **Governance:** decision rights were never defined. People genuinely don't know if a call is theirs to make, so they escalate or consult to be safe.
- **Vendor behaviour:** the SI is often happy to let client decisions drift — every week of indecision is a billable week, and the delay isn't their contractual fault.
- **Sponsorship:** a sponsor who delegates upward-feeling decisions back down creates a vacuum no committee can fill. Decision paralysis usually starts at the top.

Maturity ladder — where is your program?

Level	What it looks like in practice
1 • Critical	Key decisions open for weeks, recurring on every agenda. Owners are committees or roles, never names. Decisions made by deadline default.
2 • Fragile	A decision log exists but records discussions, not decisions. Some owners named, but no deadlines or escalation when they slip.
3 • Aware	Every open decision has one named owner and a deadline. A regular decision forum exists. Aging is visible even if escalation isn't automatic.
4 • Resilient	Decisions close at delivery pace. Aging triggers automatic escalation. Default-by-deadline decisions are treated as a process failure and reviewed.

Cost of inaction & go-live impact

If you do nothing...	Typical consequence	Rough cost signal
Design decision made late	Build rework across dependent objects	4–8 weeks rework
Decisions block test entry	UAT compressed into an impossible window	Quality / go-live risk
Go/no-go has no owner	Unready go-live by social default	Months of hypercare
Chronic indecision	Team hedges everything, rework compounds	15–30% effort waste

What an experienced recovery lead sees that the team doesn't

What the team usually misses: The decision isn't stuck because it's hard — it's stuck because no one has been given the pen. Nine times out of ten, naming a single owner with a deadline closes a 'difficult' decision within days. The difficulty was organisational, not analytical.

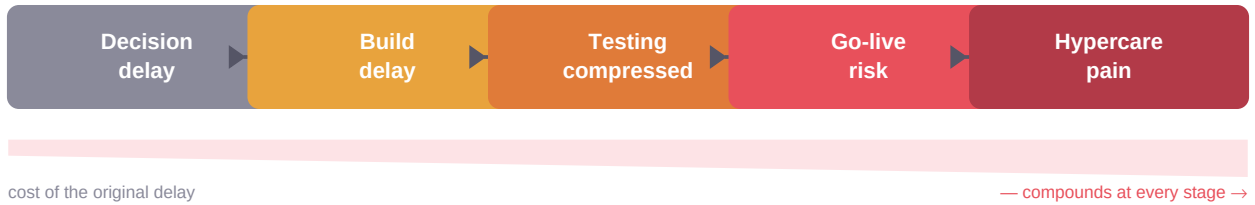
What the team usually misses: Watch where decisions are made by default — the deadline passing, the build proceeding on an assumption. Those silent defaults are more dangerous than visible stalls, because no one feels responsible for a decision nobody made.

What the team usually misses: A sponsor who says 'bring me options' on every call is often the cause, not the cure. Real recovery sometimes means telling the sponsor: the team needs you to decide this one, not to be consulted on it.

How Nobody-Wants-To-Decide breeds the other patterns

How this pattern breeds others: It feeds **Death By Governance**: the organisational response to a stuck decision is almost always to add a forum or a review to 'help it along' — more process, less progress. It deepens **Hero Dependency**: when decisions won't flow through the proper channels, they route to whoever is willing to just make the call, concentrating power and risk in one person. And it accelerates **Silent Resistance**: teams waiting on decisions that never come start building their own workarounds, which harden into the shadow processes you'll fight at go-live. Decision paralysis is rarely a standalone problem — it's usually the engine driving two or three others.

One stalled decision doesn't stay one problem. It compounds through go-live into hypercare.



This is why Nobody Wants To Decide is the first pattern to break.

Death By Governance™

SCORE 80 · CRITICAL

The program has more process than it can afford. Approvals, forums and reports consume the capacity that should be delivering. Everyone is busy; nothing moves — and every past failure has left behind a control that no one will remove.

Phrases that tell you it's here

PHRASES YOU HEAR IN THE ROOM (each one is a tell)

"Let's take it to the board next week." → A simple change just acquired a two-week latency and three sign-offs.

"We need another working group to align on this." → A new forum is being born to solve what an owner should decide.

"Has this been through change control yet?" → Asked about something trivial — the process is now reflexive, not risk-based.

"I just need a few more sign-offs." → Accountability has been spread until no one carries it.

"Let me update the deck before we discuss." → Reporting has become the work, not the assurance of the work.

What it looks like in the wild

CASE 1 Utilities, SAP S/4 — the four-board change

Situation: Every change passed through design authority, a change board, an integration forum and steering. Four forums, each weekly.

The tell: A one-line config change to a tax code took 19 days to clear. Nobody found that strange any more.

What happened: The defect backlog never cleared because triage couldn't keep pace with the governance cadence. Go-live slipped twice; the real cause — governance latency — was never named in any steering paper.

The lesson: The program didn't lack control. It was drowning in it. The cure looked reckless to the people who'd built each control for a good reason.

CASE 2 Public sector, Oracle — assurance theatre

Situation: A high-scrutiny program responded to political risk by adding assurance layers: three gates, two external reviewers, monthly board packs.

The tell: The team spent roughly 35% of capacity producing evidence for reviews rather than building. 'We're very well governed' was said often, with pride.

What happened: Delivery velocity collapsed under reporting load. The program was cancelled and re-scoped — not because it was unsafe, but because it was too slow to ever finish.

The lesson: In high-scrutiny environments, governance grows fastest and is hardest to cut, because cutting it feels like inviting the very failure it's there to prevent.

CASE 3 Manufacturing, Dynamics — the meeting that wouldn't die

Situation: A daily 'cutover sync' set up during an earlier crisis kept running for nine months after the crisis passed.

The tell: 20 people, 45 minutes, daily. When asked what decision it had made that week, no one could name one.

What happened: Standing it down freed roughly 75 person-hours a week — redirected to actual cutover prep that had been slipping. Readiness moved more in three weeks than the previous two months.

The lesson: Governance set up in a crisis must be sunset deliberately, or it becomes permanent cost long after its reason has gone.

How four people see the same problem

Seen from...	What this pattern looks like to them
CIO	Sees: 'We have strong governance and good oversight.' Reality: Oversight has tipped into drag; the CIO sees the reports, not the cost of producing them.
Sponsor	Sees: 'Why is everything taking so long to approve?' Reality: The sponsor feels the latency but rarely connects it to the controls they themselves asked for.
PM	Sees: 'I spend my week feeding forums, not delivering.' Reality: Correct — and they often can't cut the forums because they didn't create them and fear the blame if something slips.
Team	Sees: 'We prepare for meetings about work instead of doing work.' Reality: The clearest view of all. The team feels the waste daily but has no authority to remove it.

Maturity ladder — where is your program?

Level	What it looks like in practice
Critical	Trivial changes take weeks. Multiple overlapping forums re-litigate the same decisions. Reporting has become the deliverable.
Fragile	Approval chains routinely exceed three sign-offs. Forums multiply faster than they're retired. Delivery capacity visibly eroded.
Aware	The cost of governance is acknowledged; some forums questioned. Approval paths mapped but not yet cut.
Resilient	Two-signature maximum. Every forum has a purpose and a decision quota. A standing mechanism retires controls that stop earning their place.

Cost of inaction & go-live impact

If you do nothing...	Typical consequence	Rough cost signal
of team capacity lost to process & reporting		20–35%
added latency per critical decision path		2–4 wks
defect triage can't reach go-live-ready		Backlog

cutover decisions stuck in weekly cadence		Slip
---	--	------

What an experienced recovery lead sees that the team doesn't

What the team usually misses: Every control was added for a real reason — that's exactly why it's so hard to remove. The skill isn't spotting bad governance; it's having the authority and nerve to retire good governance that has outlived its risk.

What the team usually misses: Cutting a forum without naming who now owns its decisions doesn't reduce governance — it converts Death By Governance straight into Nobody Wants To Decide. Always move the decision rights to a named owner as you cut.

What the team usually misses: The most over-governed programs are usually the ones that were burned before. Fear is the fuel. You cannot cut governance by argument alone; you cut it by making one visible reduction and showing nothing bad happened.

How Death By Governance breeds the other patterns

How this pattern breeds others: When process smothers delivery, three things follow. It feeds Nobody Wants To Decide: layered approvals diffuse accountability until no single person owns anything. It masks Silent Resistance: teams comply with process on paper while quietly working around it, and the governance reports never show it. And it starves Builder Culture: people stop solving problems because every fix needs four sign-offs, so they escalate instead. Cut governance and you often release a wave of initiative the process had been suppressing.



A one-line config change takes 19 days to clear six gates. The change isn't the problem — the funnel is.

The system went live; the behaviour never changed. People nod in training, then quietly keep doing it the old way. Adoption looks fine on the report and fails on the floor — because resistance here is polite, invisible, and patient.

Phrases that tell you it's here

PHRASES YOU HEAR IN THE ROOM (each one is a tell)

"We'll use the system, but I'll keep my spreadsheet just in case." → The shadow process announced out loud. 'Just in case' means 'I don't trust this'.

"Training went really well, everyone attended." → Attendance is being reported as adoption. They are not the same thing.

"That's not how we actually do it here." → The new process and the real process have already diverged.

"Let's just get through go-live, then we'll see." → Quiet permission to revert the moment pressure rises.

"The old way was fine, honestly." → The benefits case is being rejected at the floor, politely.

What it looks like in the wild

CASE 1 Consumer goods, SAP — the parallel close

Situation: Finance went live on a new GL but quietly kept running the legacy reconciliation in spreadsheets every month-end.

The tell: Month-end close still took the same number of days as before go-live. Nobody asked why the promised improvement hadn't appeared.

What happened: Six months in, an audit found the system and the spreadsheets disagreed. The benefits case had never materialised because the behaviour never changed. A remediation program was needed to do the adoption that should have happened at go-live.

The lesson: Silent Resistance doesn't fail loudly at go-live. It fails quietly, months later, as a benefits shortfall nobody can immediately explain.

CASE 2 Pharma manufacturing, Oracle — the batch-record workaround

Situation: Shop-floor operators were given new electronic batch records but found them slower under production pressure.

The tell: Operators completed the e-records at end of shift from paper notes, rather than in real time. Compliance saw 'completed records' and was satisfied.

What happened: A regulatory audit caught the retrospective completion — a serious data-integrity finding. The fix was costly and public. The resistance had been rational: the system genuinely slowed them down.

The lesson: Silent Resistance is usually a signal, not stubbornness. The workaround exists because the new way doesn't yet beat the old way for the person doing the work.

CASE 3 Retail, Dynamics — the super-user crutch

Situation: Store managers were trained but leaned on a handful of super-users for anything beyond the basics.

The tell: Super-users were quietly doing other people's transactions 'to save time' — recreating the old division of labour inside the new system.

What happened: When super-users moved on, capability cratered because real adoption had never spread. Transaction errors and stock discrepancies spiked.

The lesson: A super-user who does the work for everyone is hiding a non-adoption problem, not solving it. Watch where the real keystrokes happen.

How four people see the same problem

Seen from...	What this pattern looks like to them
CIO	Sees: 'Adoption metrics are green — logins and transactions are up.' Reality: Vanity metrics. Volume of logins says nothing about whether the behaviour or the outcome changed.
Sponsor	Sees: 'We trained everyone; why aren't the benefits showing?' Reality: Training was delivered, not landed. The sponsor measured the input, not the behaviour change.
PM	Sees: 'It's an adoption issue, change management owns it.' Reality: Often outsourced to 'change' as if it's separable from delivery — which is why it festers.
Team	Sees: 'The new system is slower for what I actually do.' Reality: Frequently true and rarely heard. The resistance is rational and contains the fix.

Maturity ladder — where is your program?

Level	What it looks like in practice
Critical	Shadow processes everywhere. Reported adoption is green while business outcomes don't move. Data quality degrading silently.
Fragile	Workarounds common at pressure points (month-end, peak). Super-users doing others' work. Benefits case quietly slipping.
Aware	Shadow processes identified through observation. Resistance understood as signal, not defiance. Recovery plan per group forming.
Resilient	Old paths closed where the new one works. Competence measured, not attendance. Resistant groups converted to advocates through quick wins.

Cost of inaction & go-live impact

If you do nothing...	Typical consequence	Rough cost signal
benefits realised despite 'successful' go-live		0%
becomes a recurring crisis as people revert		Month-end

quality erodes silently, surfaces months later		Data
a remediation adoption program is needed		Re-do

What an experienced recovery lead sees that the team doesn't

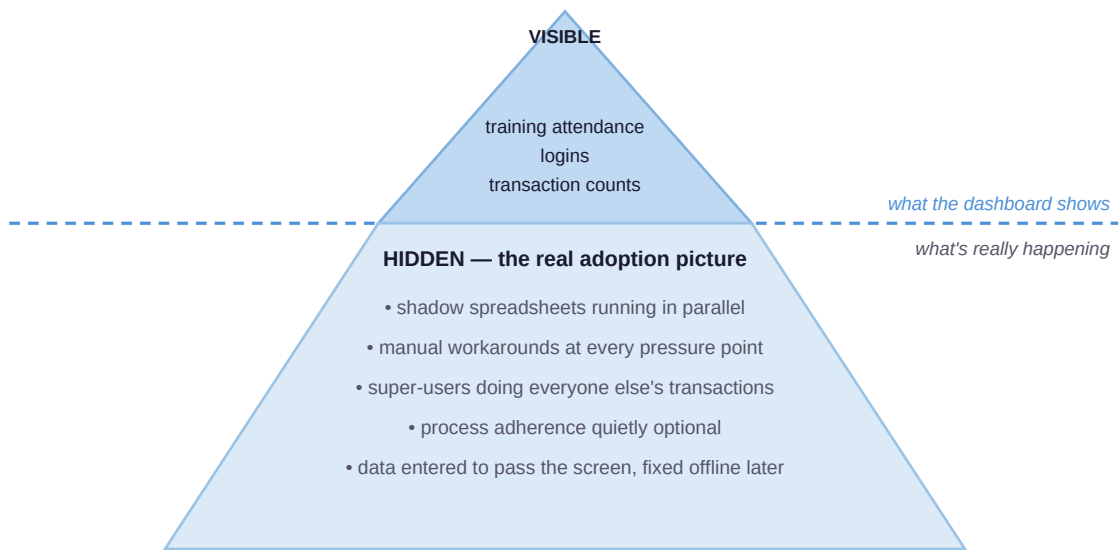
What the team usually misses: Green adoption dashboards are the most dangerous artefact in a transformation. Logins and transaction counts can stay perfect while the entire business quietly routes around the system. Measure outcomes — close times, error rates, benefit realisation — not activity.

What the team usually misses: Almost every workaround is rational from the worker's seat. The instinct to police it is wrong; the move is to ask what the workaround gives them that the system doesn't, then close that gap. Resistance is unmet need wearing a disguise.

What the team usually misses: Never remove the old way before the new way genuinely works. Kill the shadow path too early and you cause a real failure that proves the resisters right and hardens the resistance. Sequence: prove value, then remove the alternative.

How Silent Resistance breeds the other patterns

How this pattern breeds others: Quiet non-adoption rarely stays contained. It manufactures Hero Dependency: when most people work around the system, the few who truly know it become indispensable crutches. It feeds Nobody Wants To Decide: leaders won't make the hard call to close the old way, so the decision drifts. And it can be hidden by Death By Governance, whose green compliance reports mask the behaviour underneath. The deepest cause is usually a weak Builder Culture — a team that owned the change would have surfaced the friction instead of routing around it silently.



Green adoption metrics measure the tip. The resistance that kills your benefits case lives below the line.

The healthy pattern — the one you protect and grow, not cure. In a builder culture, problems get solved instead of escalated. When it's weak, the other four patterns grow back no matter how many times you cut them down. This is the pattern that keeps the others fixed.

Phrases that tell you about its health

PHRASES YOU HEAR IN THE ROOM (each one is a tell)

"I hit a blocker, worked two options, went with the second — here's why." → STRONG. A builder reporting a solved problem, not asking permission.

"That's not my area, I've raised a ticket." → WEAK. The problem is visible but ownership is being deflected upward.

"I wasn't sure I was allowed to fix it, so I waited." → WEAK. Authority and initiative have been trained out of the team.

"We tried something, it didn't work, here's what we learned." → STRONG. Safe-to-fail behaviour — the engine of a builder culture.

"Let's wait for the steering committee to tell us." → WEAK. Learned helplessness, dressed as good governance.

What it looks like in the wild

CASE 1 Industrial, SAP — the team that recovered itself

Situation: A mid-sized rollout hit a serious data-migration defect three weeks before go-live.

The tell: The team didn't wait for a crisis meeting. Two leads worked the weekend, built a reconciliation fix, and presented it Monday already tested.

What happened: Go-live held. In the retro, leadership realised the recovery happened because they'd spent months rewarding exactly this behaviour. The culture was the contingency plan.

The lesson: Builder Culture is invisible until pressure hits — then it's the difference between a self-correcting team and a program that escalates its way into failure.

CASE 2 Services firm, Oracle — the culture that was quietly killed

Situation: A capable team gradually stopped solving problems after a new governance regime punished a well-intentioned fix that went slightly wrong.

The tell: Within two months, everything became a ticket and a meeting. The phrase 'let's not get ahead of governance' started appearing.

What happened: Velocity halved. The same people who'd been proactive became passive, waiting to be told. The program needed a hero to rescue it — Builder Culture's collapse had created Hero Dependency.

The lesson: Builder Culture is fragile. One public punishment of a good-faith fix can switch a whole team from solving to waiting. Psychological safety is the substrate.

How four people see it

Seen from...	What this pattern looks like to them
CIO	Sees: 'I want an empowered, proactive team.' Reality: Often genuinely wanted — and unknowingly undermined by the very controls and metrics the CIO also asks for.
Sponsor	Sees: 'Why does everything escalate to me?' Reality: Because authority was never devolved. The team escalates because they've learned they must.
PM	Sees: 'I can't get the team to take ownership.' Reality: Ownership can't be requested while authority is withheld. The PM may be asking for initiative inside a system that punishes it.
Team	Sees: 'It's safer to ask than to act.' Reality: The honest truth. In a low-safety environment, waiting is rational self-protection.

Maturity ladder — where is your program?

Level	What it looks like in practice
Critical	Everything escalates. The team waits to be told. Problems become tickets and meetings, never fixes. No psychological safety.
Fragile	Initiative is sporadic and person-dependent. Good fixes happen but aren't rewarded, so they don't spread. Heroics celebrated over building.
Aware	Problem-solving is recognised when it happens. Some decision rights devolved. A forum exists for the team to solve together.
Resilient	Teams resolve a defined class of issues without escalation. Builder behaviour is named, rewarded and protected — especially under go-live pressure.

Cost of a weak Builder Culture

If you do nothing...	Typical consequence	Rough cost signal
the other four patterns regrow after every fix		Recurring
every issue escalates; command structure jams		Overload
small problems become program-level under pressure		Crisis
no self-correction through cutover & hypercare		Fragile

What an experienced recovery lead sees that the team doesn't

What the team usually misses: Builder Culture is the only pattern you grow rather than cut — and it's the one that determines whether the other four stay fixed. Recoveries that ignore it relapse, because the conditions that created the dysfunction are still intact.

What the team usually misses: Celebrating the hero who saved the day actively erodes Builder Culture — it teaches everyone that rescue, not prevention, is what gets rewarded. Celebrate the team that quietly prevented the crisis nobody saw. That's the harder, better signal.

What the team usually misses: Under go-live pressure, leaders instinctively revert to command-and-control and undo months of culture-building in a week. Protect Builder Culture hardest exactly when the pressure is highest — that's when you most need a team that self-corrects.

How Builder Culture governs the other four patterns

How this pattern breeds others: Builder Culture is the master pattern — its weakness is what lets the others persist. A strong builder culture dissolves Hero Dependency (the team shares load instead of crowning a hero), resists Death By Governance (people fix rather than escalate, so process stays lean), prevents Nobody Wants To Decide (ownership is instinctive), and pre-empts Silent Resistance (a team that owns the change surfaces friction instead of hiding it). This is why recovery sequencing ends here: cut the other four down, but grow this one, or they all return.

Using the five together

The patterns are not independent — they feed each other, which is why fixing one in isolation rarely holds. The interaction blocks throughout this guide map the specific mechanisms. The practical sequence that works across most recoveries:

- Start where the live work is heaviest — the Pattern Dashboard in your workbook shows this by counting the risks, decisions, people and actions tagged to each pattern.
- Unblock **Nobody Wants To Decide** early: stuck decisions gate everything downstream, so closing them accelerates every other recovery.
- Break **Hero Dependency** before cutover: it is the single most predictive pattern for go-live failure.
- Cut **Death By Governance** to release the team capacity you need for everything else — but move decision rights to named owners as you cut.
- Work **Silent Resistance** steadily through peers and quick wins; it is the slowest to move and the easiest to fake green.
- Grow **Builder Culture** throughout — it is what stops the other four growing back once you have cut them down.

What the team usually misses: If you remember one thing: you are not fixing five separate problems. You are shifting a system. Builder Culture is the lever that holds the shift in place — which is why every durable recovery ends by growing it, not just by cutting the other four.

PMOS™ · Pattern Recovery Field Guide · Consultant Edition · Behavioral Diagnosis & Recovery for ERP and Transformation Programs